

1.3 Boucle 3

Soit la boucle Soit la boucle 3 ci-dessous

```
DO I = 2, 100
  DO J = 2, 100
    A(I,J) = A(I,J-1) + A(I,J+1) + A(I-1,J) + A(I+1,J)
  ENDDO
ENDDO
```

BOUCLE 3

- 1) Calculer les dépendances au sein de la double boucle (Type, Distance)
- 2) Que vous indique le test du PGCD pour la parallélisation de la boucle la plus interne ?
- 3) En revenant aux équations de dépendances, est ce que la boucle la plus interne est parallélisable ?
- 4) Est-ce que la boucle la plus externe est parallélisable
- 5) Est-ce que les 2 boucles peuvent être échangées/interverties (« interchange ») ?
- 6) Identifier des ensembles d'itérations qui peuvent être exécutées simultanément

1) Dépendances dans la double boucle (Type, Distance)

On écrit $A(i, j)$ et on lit $A(i, j - 1), A(i, j + 1), A(i - 1, j), A(i + 1, j)$.

- Sur j (i fixé) :
 - RAW : $(i, j - 1) \rightarrow (i, j)$ via $A(i, j - 1)$, distance = 1.
 - WAR : $(i, j) \rightarrow (i, j + 1)$ via $A(i, j + 1)$, distance = 1.
- Sur i (j fixé) :
 - RAW : $(i - 1, j) \rightarrow (i, j)$ via $A(i - 1, j)$, distance = 1.
 - WAR : $(i, j) \rightarrow (i + 1, j)$ via $A(i + 1, j)$, distance = 1.

2) Test du PGCD pour la boucle la plus interne (sur j)

On cherche des égalités d'adresses entre une écriture (K, L) (écrit $A(K, L)$) et une lecture (M, N) .

Voisinage en j (termes $A(i, j \pm 1)$) :
 $(K, L) = (M, N - 1)$ ou $(K, L) = (M, N + 1)$
 $\Rightarrow K - M = 0, \quad L - N = \pm 1$

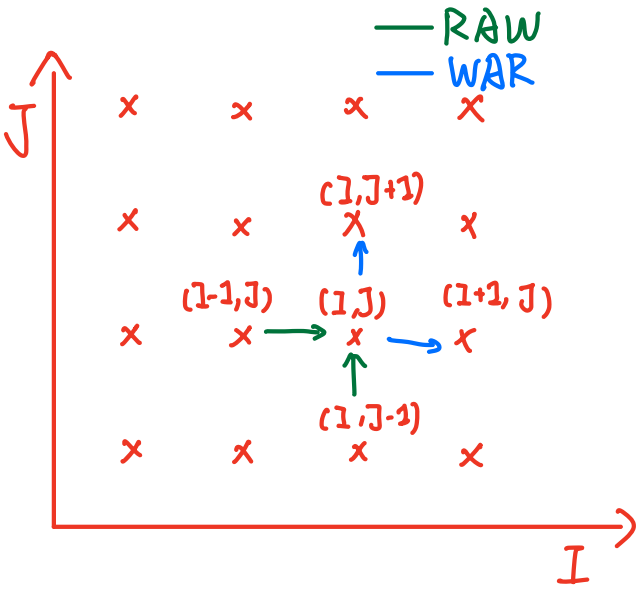
$\gcd(1, -1) = 1$ divise $\pm 1 \Rightarrow$ test du PGCD positif
Donc il y a une possibilité de dépendance sur j (le test ne l'élimine pas).

3) La boucle la plus interne est-elle parallélisable ?

Non : les équations ci-dessus admettent des solutions dans le domaine ($j \pm 1 \in [2, 100]$ pour j intérieur), et on a une dépendance RAW effective $(i, j - 1) \rightarrow (i, j)$.
Donc la boucle interne j n'est pas parallélisable.

4) La boucle la plus externe est-elle parallélisable ?

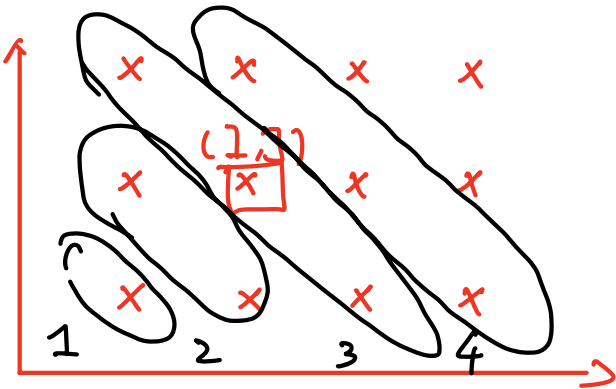
Non : il existe une dépendance RAW portée par i via $A(i - 1, j) : (i - 1, j) \rightarrow (i, j)$ (distance 1).
Donc la boucle externe i n'est pas parallélisable.



5)

Oui, l'échange est valide : pour un i fixé, l'ordre des lectures et écritures selon j est conservé, et les itérations de lignes différentes n'interfèrent pas.

6)



Les itérations appartenant à un même ensemble tel que $i + j = \text{constante}$ peuvent être exécutées simultanément.
En effet, il n'existe aucune dépendance entre deux itérations d'un tel ensemble.

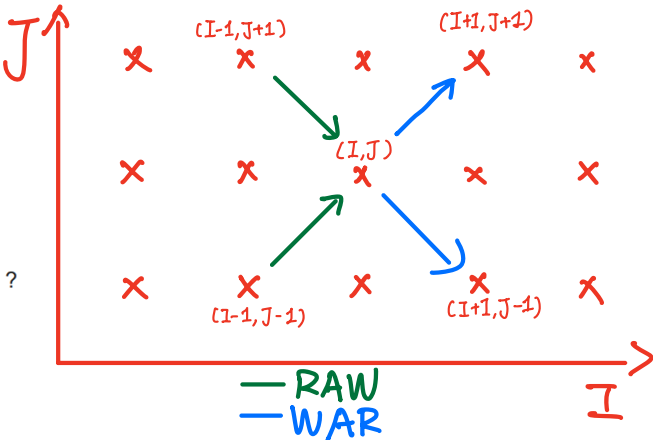
1.4 Boucle 4

Soit la boucle 4 ci-dessous

```
DO I = 2, 100
  DO J = 2, 100
    A(I,J) = A(I-1,J-1) + A(I-1,J+1) + A(I+1,J-1) + A(I+1,J+1)
  ENDDO
ENDDO
```

BOUCLE 4

- 1) Calculer les dépendances au sein de la double boucle (Type, Distance)
- 2) Que vous indique le test du PGCD pour la parallélisation de la boucle la plus interne ?
- 3) En revenant aux équations de dépendances, est ce que la boucle la plus interne est parallélisable ?
- 4) Est-ce que la boucle la plus externe est parallélisable
- 5) Est-ce que les 2 boucles peuvent être échangées/interverties (« interchange ») ?
- 6) Identifier des ensembles d'itérations qui peuvent être exécutées simultanément



1.4 Boucle 4

1) Calculer les dépendances au sein de la double boucle (Type, Distance)

On écrit $A(I, J)$ et on lit $A(I-1, J-1)$, $A(I-1, J+1)$, $A(I+1, J-1)$ et $A(I+1, J+1)$.

Le parcours se fait par I croissant, puis J croissant.

- Sur $I-1$ (itérations passées) :
 - RAW : $(I-1, J-1) \rightarrow (I, J)$ via $A(I-1, J-1)$, $d_I = 1$, $d_J = 1$
 - RAW : $(I-1, J+1) \rightarrow (I, J)$ via $A(I-1, J+1)$, $d_I = 1$, $d_J = -1$
- Sur $I+1$ (itérations futures) :
 - WAR : $(I, J) \rightarrow (I+1, J-1)$ via $A(I+1, J-1)$, $d_I = 1$, $d_J = -1$
 - WAR : $(I, J) \rightarrow (I+1, J+1)$ via $A(I+1, J+1)$, $d_I = 1$, $d_J = 1$

“Conclusion Q1 : dépendances RAW et WAR, toutes portées par I avec $d_I = \pm 1$ (donc aucune avec $d_I = 0$).”

2) Que vous indique le test du PGCD pour la parallélisation de la boucle la plus interne ?

On cherche des égalités d'adresses entre une écriture (K, L) et une lecture (M, N) sur l'indice J :

$$K = I, L = J$$
$$M = I \pm 1, N = J \pm 1$$

Voisinage en J : $L = N \pm 1 \Rightarrow J = J \pm 1 \Rightarrow 1 \cdot J - 1 \cdot J = \pm 1$.

$PGCD(1, -1) = 1$, et 1 divise $\pm 1 \Rightarrow$ test du PGCD positif.

“Le test du PGCD est **non concluant** : il n'élimine pas une dépendance, mais **ne permet pas** non plus d'affirmer qu'elle est portée par J (il faut revenir aux équations complètes).”

3) “En particulier, il n'existe **aucune dépendance avec** $d_I = 0$, donc **aucune dépendance** portée par J .”

En observant les distances, on remarque que toutes les dépendances ont une distance sur I égale à 1 (ou -1). Il n'existe aucune dépendance avec une distance sur I nulle.

Cela signifie que pour un I fixé, toutes les itérations de J peuvent être exécutées en même temps sans conflit.

Donc la boucle interne J est parallélisable.

2 BOUCLE SOR 2D

Dans le code ci-dessous (voir page suivante) extrait de Numerical Recipes, analysez la quadruple boucle.

- 1) Précisez les accès au tableau $u[j][l]$ dans les 2 boucles internes (boucles j et l)
- 2) Quels sont les rôles des variables lsw , jsw , $ipass$?
- 3) Quel est le rôle de la boucle sur $ipass$?

1. Accès au tableau $u[j][l]$ dans les 2 boucles internes (j et l)

Dans le cœur :

- Boucle for ($j=2$; $j<jmax$; $j++$) : on parcourt les lignes $j = 2..jmax-1$.
- Boucle for ($l=lsw+1$; $l<jmax$; $l+=2$) : on parcourt **une colonne sur deux** (pas 2), donc soit les 1 impairs soit les 1 pairs selon lsw .

À chaque itération (j, l) on calcule un résidu (équation 5-points) et on met à jour le point central :

- Lecture des voisins + soi-même :
 - $u[j+1][l]$, $u[j-1][l]$ (haut/bas)
 - $u[j][l+1]$, $u[j][l-1]$ (droite/gauche)
 - $u[j][l]$ (centre)
- Écriture :
 - $u[j][l] -= \text{omega} * \text{resid} / e[j][l]$ (mise à jour SOR au même point)

Donc, par construction, dans la boucle l , on ne touche jamais deux fois au même $u[j][l]$, et on n'écrit que sur une “couleur” (un sous-ensemble alterné) à la fois.

2. Rôle de lsw , jsw , $ipass$

- $ipass$: index de **pas**se (1 puis 2). Chaque passe met à jour **une des deux couleurs** du damier (rouge puis noir, ou l'inverse).
- jsw : “switch” de **parité entre les passes**. À la fin d'une passe, $jsw = 3-jsw$ bascule $1 \leftrightarrow 2$, ce qui décale le motif pair/impair pour la passe suivante.
- lsw : “switch” de **parité dans une passe, ligne par ligne**. On initialise $lsw = jsw$, puis après chaque ligne j on fait $lsw = 3-lsw$ pour alterner le point de départ sur la ligne suivante. Ça fabrique le motif **échiquier** (checkerboard) sur la grille 2D.

4) Est-ce que la boucle la plus externe est parallélisable ?

Non.

“Il existe des dépendances **RAW (et aussi WAR)** portées par I (toutes avec $d_I = 1$).”

Donc la boucle externe I n'est pas parallélisable.

Q5) Est-ce que les 2 boucles peuvent être échangées ?

- Non.
- Explication (Logique de cohérence) : Dans le code original, l'itération (I, J) lit la valeur de $A(I-1, J+1)$ une fois que celle-ci a été mise à jour par l'itération précédente sur I .
- Si on inverse les boucles (J en extérieur), on traite les colonnes une par une. Au moment de calculer la colonne J , la valeur de $A(I-1, J+1)$ n'a pas encore été calculée (car elle appartient à la colonne suivante $J+1$).
- Résultat : L'ordre des dépendances n'est plus respecté et le résultat final sera différent du code séquentiel original. L'interchange est donc invalide.

b) Les itérations peuvent être regroupées **par lignes**. Pour un I fixé, **toutes** les itérations (I, J) avec $J = 2..100$ peuvent s'exécuter simultanément, car il n'existe **aucune dépendance portée par J** (toutes les dépendances ont $d_J \neq 0$). En revanche, on doit respecter l'ordre entre lignes : la ligne I ne peut commencer qu'après la fin de la ligne $I-1$ (dépendances portées par I).

```
#include <math.h>
#define MAXITS 1000
#define EPS 1.0e-5

void sor(double **a, double **b, double **c, double **d, double **e,
double **f, double **u, int jmax, double rjac)
Successive overrelaxation solution of equation (19.5.25) with Chebyshev acceleration. a, b, c,
d, e, and f are input as the coefficients of the equation, each dimensioned to the grid size
[1..jmax][1..jmax]. u is input as the initial guess to the solution, usually zero, and returns
with the final value. rjac is input as the spectral radius of the Jacobi iteration, or an estimate
of it.
{
void nrerror(char error_text[]);
int ipass,j,jsw,l,lsw,n;
double anorm,anormf=0.0,omega=1.0,resid;
Double precision is a good idea for jmax bigger than about 25.

for (j=2;j<jmax;j++)
Compute initial norm of residual and terminate iteration when norm has been reduced by
a factor EPS.
for (l=1;l<jmax;l++)
anormf += fabs(f[j][l]); Assumes initial u is zero.
for (n=1;n<=MAXITS;n++) {
anorm=0.0;
jsw=1;
for (ipass=1;ipass<=2;ipass++) { Odd-even ordering.
lsw=jsw;
for (j=2;j<jmax;j++) {
for (l=lsw+1;l<jmax;l+=2) {
resid=a[j][l]*u[j+1][l]
+b[j][l]*u[j-1][l]
+c[j][l]*u[j][l+1]
+d[j][l]*u[j][l-1]
+e[j][l]*u[j][l]
-f[j][l];
anorm += fabs(resid);
u[j][l] -= omega*resid/e[j][l];
}
lsw=3-lsw;
}
jsw=3-jsw;
omega=(n == 1 && ipass == 1 ? 1.0/(1.0-0.5*rjac*rjac) :
1.0/(1.0-0.25*rjac*rjac*omega));
}
if (anorm < EPS*anormf) return;
}
nrerror("MAXITS exceeded");
}
```

3. Rôle de la boucle sur ipass

La boucle `for (ipass=1; ipass<=2; ipass++)` réalise explicitement l'**ordonnancement rouge-noir** :

- Passe 1 : mise à jour de tous les points d'une couleur (par ex. "rouge") : on peut les mettre à jour sans écrire sur leurs voisins immédiats, car leurs voisins sont de l'autre couleur.
- Passe 2 : mise à jour de l'autre couleur (par ex. "noir"), en utilisant les valeurs éventuellement déjà rafraîchies de la première couleur.

C'est l'astuce classique pour rendre un schéma type Gauss–Seidel/SOR **plus parallélisable** : à l'intérieur d'une même couleur, les points sont (idéalement) indépendants entre eux, puisque leurs dépendances directes portent sur l'autre couleur.